

Arbitrary Precision Arithmetic

Project

Yanamala Shamili
CS24BTECH11067

Departement of CSE
IIT HYDERABAD

May 2025

Introduction

In most modern programming languages, dealing with numbers is straightforward—until the numbers become exceptionally large or require high decimal precision. Standard data types such as `int`, `long`, or `float` have limited range and accuracy. When these limits are exceeded, errors occur.

To handle such cases, we use arbitrary-precision arithmetic. The goal of this project is to go beyond the built-in limits of number handling in Java. The aim is to build a system that can perform arithmetic operations—such as addition, subtraction, multiplication, and division—on very large numbers without relying on any built-in classes like `BigInteger` or `BigDecimal`. Instead, the system uses strings and custom logic to achieve this functionality.

In this project, I have gained a deeper understanding of Object-Oriented Programming (OOP).

Abstract:

This project presents a Java-based arbitrary-precision arithmetic library that enables arithmetic operations on very large numbers beyond the limits of standard data types like `int` and `float`. The core of this system is the manipulation of numbers as strings, with operations performed digit-by-digit. Special attention is given to handling negative values and ensuring decimal precision. Additionally, this project includes a Python script to automate the compilation and execution of operations. Build automation is managed using Ant, similar to a Makefile, to simplify usage and streamline the development process.

1 Project Directory Structure

The project follows a structured layout to separate source code, compiled classes, and build tools. Below is the directory structure:

```
arbitrary-arithmetic/  
| build/  
|   arbitraryarithmetic/  
|       AFloat.class  
|       AInteger.class  
|       MyInfArith.class  
  
| src/
```

```

| arbitraryarithmetic/
|   AFloat.java
|   AInteger.java
|   MyInfArith.java
| build.xml
| run_project.py

```

2 Overview:

- `AIInteger.java` - Implements large integer arithmetic.
- `AFloat.java` - Implements high precision floating point arithmetic.
- `MyInfArith.java` - Main class to perform the operations.
- `build.xml` - Apache Ant script for compiling the project with ease.
- `run_project.py` - Python script to run the Java code from Python.

2.1 `AIInteger.java`

- Stores the numbers as strings for unlimited size.
- This class is designed to perform arbitrary precision arithmetic on integers, supporting operations such as addition, subtraction, multiplication, and division.
- The class maintains a boolean flag to track the sign of the number, allowing it to correctly handle both positive and negative integers.
- The default constructor initializes the number to "0" with a positive sign.
- The string constructor initializes the number from a string, determining if it is negative based on the first character.
- The copy constructor creates a new instance from another `AIInteger` object.
- `parse()` - Parses a string to create an `AIInteger` object.
- `removeLeadingZeros()` - Removes leading zeros from a string.
- `comparing()` - Compares the current number with another `AIInteger`.
- Sign-aware public methods:
 - `add()`, `subtract()`, `multiply()`, `division()`.
- Signs are handled based on the requirements of the operation.

2.2 AFloat.java

- Stores the floating-point numbers as strings.
- Checks the sign using the boolean flag `is.negative`.
- The default constructor initializes the number to "0.0".
- The string constructor initializes the number from a string and determines its sign.
- The copy constructor creates a new instance from another `AFloat` object.
- `parse()` - Parses a string to create an `AFloat` object.
- `removeLeadingZeros()` - Removes leading zeros from a string.
- `comparing()` - Compares the current number with another `AFloat`.
- Operations:
 - `add()`, `subtract()`, `multiply()`, `division()`.
- In each of the operation methods, the decimal numbers and integers are separated to simplify calculations.

2.3 MyInfArith.java

- Accepts command-line arguments in the format: `<type> <operation> <num1> <num2>`.
- Based on the `<type>` (either `int` or `float`), the inputs `num1` and `num2` are parsed into either `AInteger` or `AFloat`.
- Performs the required arithmetic operation and prints the result.
- The logic for operation handling is implemented using `switch-case` statements.

2.4 build.xml

- Apache Ant script to automate the build process.
- Targets:
 - `clean`: Deletes the `build/` directory.
 - `init`: Creates the `build/` directory if it doesn't exist.
 - `compile`: Compiles Java files from `src/` to `build/` (depends on `clean` and `init`).
 - `jar`: Creates a JAR file from the compiled classes.
 - `run`: Runs the compiled Java classes.

2.5 run_project.py

- Compiles Java files using `javac`.
- Uses the following Python modules:
 - `sys`: Interacts with command-line arguments.
 - `os`: Interacts with the operating system to run shell commands.
- `build_project()`:
 - Runs the Ant build command to compile the Java project.
 - Checks if the build was successful and exits with an error code if not.
- `run_java(arg)`:
 - Runs the Java program using `java -cp build arbitraryarithmetic.MyInfArith` with the provided command-line arguments.
 - Checks if the Java program executed successfully and exits with an error code if not.
- `main()`:
 - Parses and validates command-line arguments (`type`, `operation`, `num1`, `num2`).
 - Calls `build_project()` to compile the project.

3 Test Cases

- `java -cp build arbitraryarithmetic.MyInfArith int add 23650078224912949497310933240250 42939783262467113798386384401498`
 - Output: 66589861487380063295697317641748
- `java -cp build arbitraryarithmetic.MyInfArith int sub 3116511674006599806495512758577 57745242300346381144446453884008`
 - Output: -54628730626339781337950941125431
- `java -cp build arbitraryarithmetic.MyInfArith int mul 14344163160445929942680697312322 23017167694823904478474013730519`
 - Output: 330162008905899217578310782382075660760972861550182008086155118
- `java -cp build arbitraryarithmetic.MyInfArith float add 84486723.420039 70974199.843732`
 - Output: 155460923.263771
- `java -cp build arbitraryarithmetic.MyInfArith int div 25 123`

- Output: 0
- `java -cp build arbitraryarithmetic.MyInfArith float div 5.5 2`
 - Output: 2.75
- `python3 run_project.py int add 123456 7890`
 - Output: 131346

4 Challenges

- Handling the manual carry during addition and borrowing during subtraction.
- Dealing with the sign of the numbers, especially when performing operations like addition, subtraction, and division.
- Division was more complicated than multiplication or addition due to the need to handle precision and multiple steps.
- In subtraction, special care with signs was taken to determine whether the result should involve addition or subtraction.
- In `AFloat`, for each of the methods, I had to separate the decimal numbers and the integer numbers to ensure easier calculations and maintain precision.
- Long division for floats was considered with precision up to 30 decimal digits, which added complexity to the calculations.
- The compilation time for division operations was significantly longer compared to other operations due to the increased precision required.

5 Limitations

- Division is slow due to the repeated subtraction method, which increases the time complexity for large numbers.
- Basic arithmetic algorithms are not optimized for performance when handling very large numbers, leading to slower execution.
- Storing large numbers as strings can result in high memory consumption, especially for operations on extremely large numbers.

6 Key learnings:

- Arbitrary precision logic teaches a deep understanding of number operations.
- Handling signs and some cases is important and I have understood its logic.
- Enhanced my skills in building systems from scratch.

7 Conclusion

This project gave me the opportunity to dive into how arithmetic operations really work, beyond just using standard libraries. By building everything from scratch, I learned a lot about how numbers are handled, especially when it comes to things like arbitrary-precision arithmetic. Although the system might not be the fastest, it lays a solid foundation that can be improved and expanded in the future. This project has helped me understand and solve the problem from scratch.

8 Github account:

<https://github.com/YShamili>